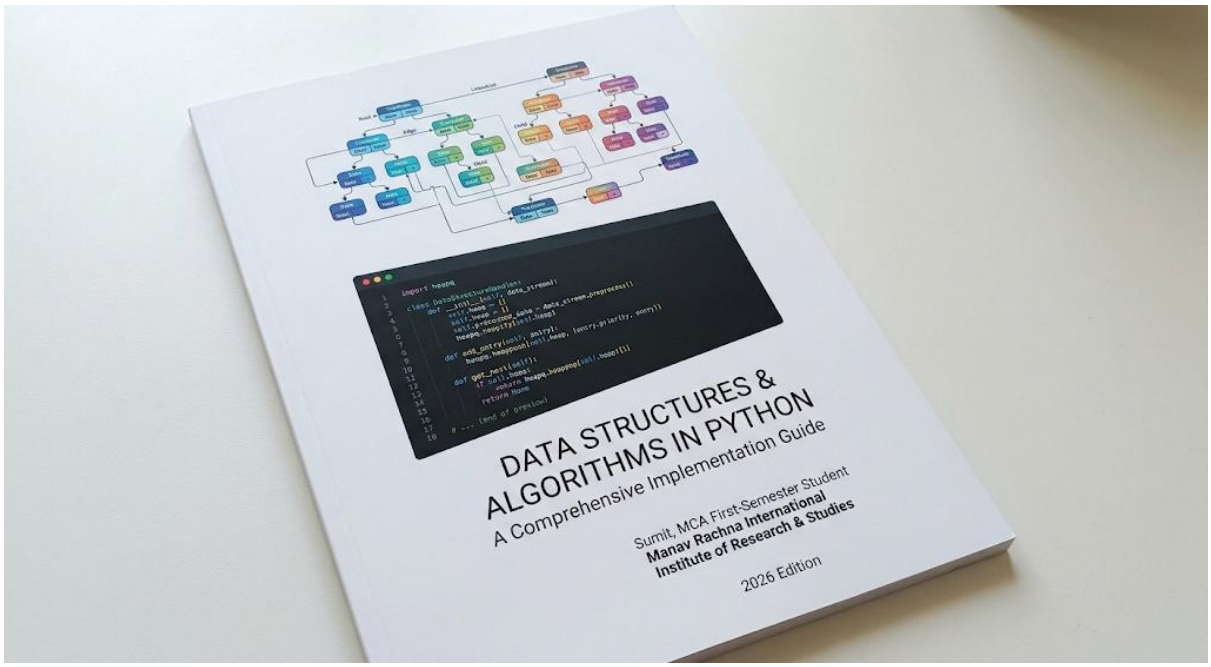
DSA_LAB_EXPERIMENT



Experiment – 1: -

Make a program for electricity bill through the unit ?

```python
# Electricity Bill

unit = int(input("Enter electricity units: "))

if unit <= 100:
    bill = unit * 3
elif unit > 101 and unit <= 300:
    bill = unit * 4
else:
    bill = unit * 5

print("Electricity bill =", bill)
```

Output:

```
neDrive/Desktop/Assigmnet/Drishti_Ds_lab/Basic.py
Enter electricity units: 67
Electricity bill = 201
```

Experiment – 2 :

Write the program to check the number is odd or even ?

```
#Check even or odd number

n = int(input("Enter a number: "))

if n % 2 == 0:
    print("Number is Even")
else:
    print("Number is Odd")
```

Output:

```
Enter a number: 4
Number is Even
```

Expriment – 3: -

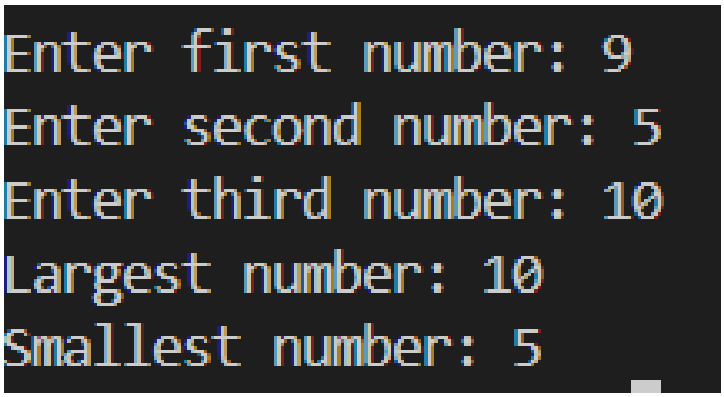
Write a program to find the greatest and smallest number?

```python
#Find Largest and Smallest of 3 Numbers

a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
c = int(input("Enter third number: "))

if a > b and a > c:
    largest = a
elif b > c:
    largest = b
else:
    largest = c
if a < b and a < c:
    smallest = a
elif b < c:
    smallest = b
else:
    smallest = c
print("Largest number:", largest)
print("Smallest number:", smallest)
```

Output : -



Experiment 5 : -

Write a program to gave the grade

```python
#Grade of Student According to Marks\
marks = int(input("Enter student marks: "))

if marks >= 90 and marks <= 100:
    print("Excellent")
elif marks >= 70 and marks < 90:
    print("Very Good")
elif marks >= 60 and marks < 70:
    print("Good")
elif marks >= 50 and marks < 60:
    print("Fair")
elif marks >= 0 and marks < 50:
    print("Poor")
else:
    print("Invalid marks")
```

Output : -

```
Enter student marks: 98
Excellent
```

Experiment – 6 :-

Make a program for insertion and deletion of array

rishti_Ds_lab > Insertion_Del.py > ...

```python
# Insertion and deletion of a Array


n = int(input("Enter the array size : "))

arr = [0]*(n + 1)

for i in range(n):
    arr[i] = int(input("Enter the element : "))

print("Original array:", arr[:n])

print("1. Insertion")
print("2. Deletion")
choice = int(input("Enter your choice : "))

if choice == 1:
    element = int(input("Enter element to insert : "))
    position = int(input("Enter position : "))

    if position < 0 or position > n:
        print("Invalid position")
    else:
        i = n
        while i > position:
            arr[i] = arr[i - 1]
            i = i - 1

        arr[position] = element
        n = n + 1

        print("Array after insertion:", arr[:n])

elif choice == 2:
    position = int(input("Enter position to delete : "))

    if position < 0 or position >= n:
        print("Invalid position")
    else:
        i = position
        while i < n - 1:
            arr[i] = arr[i + 1]
            i = i + 1

        n = n - 1
        print("Array after deletion :", arr[:n])

else:
    print("Invalid choice")
```

Output : -

```
Enter the array size : 3
Enter the  element : 2
Enter the  element : 3
Enter the  element : 4
Original array: [2, 3, 4]
1. Insertion
2. Deletion
Enter your choice : 1
Enter element to insert : 6
Enter position : 2
Array after insertion: [2, 3, 6, 4]
```

Experiment – 7 :

Write a program to check the product of 2 matrices

Drishti_Ds_lab > Product_two_m.py > ...

```python
# Program to multiply two matrices

n = int(input("Enter the matrix size : "))
a = []
b = []
result = []

print("Enter the first matrix :")
for i in range(n):
    row = []
    for j in range(n):
        row.append(int(input()))
    a.append(row)

print("Enter the second matrix : ")
for i in range(n):
    row = []
    for j in range(n):
        row.append(int(input()))
    b.append(row)

for i in range(n):
    row = []
    for j in range(n):
        total = 0
        for k in range(n):
            total = total + a[i][k] * b[k][j]
        row.append(total)
    result.append(row)

print("Product of matrices :")
for row in result:
    print(row)
```

Output : -

```
PS C:\Users\DELL\OneDrive\Desktop\Assigmnet> & C:\Users\DELL\AppData\L
neDrive/Desktop/Assigmnet/Drishti_Ds_lab/Product_two_m.py
3
5
6
7
8
7
6
Enter the second matrix :
7
8
7
8
9
7
5
7
5
Product of matrices :
[38, 47, 36]
[118, 143, 112]
```

Experiment – 8 : -

Drishti_Ds_lab > Circukar_queue.py > ...

```python
n = int(input("Enter queue size: "))
queue = [0] * n
front = -1
rear = -1

while True:
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Display")
    print("4. Exit")

    ch = int(input("Enter your choice: "))

    if ch == 1:
        if (rear + 1) % n == front:
            print("Queue is full")
        else:
            x = int(input("Enter element: "))
            if front == -1:
                front = 0
            rear = (rear + 1) % n
            queue[rear] = x

    elif ch == 2:
        if front == -1:
            print("Queue is an empty")
        else:
            print("Deleted element:", queue[front])

            if front == rear:
                front = -1
                rear = -1
            else:
                front = (front + 1) % n

    elif ch == 3:
```

Output : -

```
Enter queue size: 3
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter element: 2
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
Deleted element: 2
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 4
```

Experiment – 8 :

```python
# General Queue Program
n = int(input("Enter queue size: "))
queue = [0] * n
front = -1
rear = -1

while True:
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Display")
    print("4. Exit")

    ch = int(input("Enter your choice: "))

    if ch == 1:
        if rear == n - 1:
            print("Queue is full")
        else:
            x = int(input("Enter element: "))
            if front == -1:
                front = 0
            rear = rear + 1
            queue[rear] = x

    elif ch == 2:
        if front == -1 or front > rear:
            print("Queue is empty")
        else:
            print("Deleted element:", queue[front])
            front = front + 1

            if front > rear:
                front = -1
                rear = -1
```

```
            print("Queue is empty")
        else:
            print("Deleted element:", queue[front])
            front = front + 1

            if front > rear:
                front = -1
                rear = -1

    elif ch == 3:
        if front == -1:
            print("Queue is empty")
        else:
            for i in range(front, rear + 1):
                print(queue[i], end=" ")
            print()

    elif ch == 4:
        break

    else:
        print("Invalid choice")
```

Output :.

```
Enter your choice: 1
Enter element: 3
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter element: 3
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter element: 4
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
Deleted element: 3
1. Enqueue
2. Dequeue
3. Display
```

Experiment – 10 :

STACK

```python
# Stack operation

stack = []

while True:
    print("1. Push")
    print("2. Pop")
    print("3. Display")
    print("4. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        x = int(input("Enter element: "))
        stack.append(x)
        print("Element pushed")

    elif choice == 2:
        if len(stack) == 0:
            print("Stack is empty")
        else:
            print("Deleted element:", stack.pop())

    elif choice == 3:
        print("Stack:", stack)

    elif choice == 4:
        break

    else:
        print("Invalid choice")


# Reverse of string using stack
```

```python
# Reverse of string using stack

s = input("Enter a string: ")
stack = []

for ch in s:
    stack.append(ch)

rev = ""

while len(stack) > 0:
    rev = rev + stack.pop()
print("Reverse string:", rev)


# Matching Parentheses using stack

s = input("Enter an expression: ")
stack = []
balanced = True

for ch in s:
    if ch == '(':
        stack.append(ch)

    elif ch == ')':
        if len(stack) == 0:
            balanced = False
            break
        else:
            stack.pop()

if len(stack) != 0:
    balanced = False
if balanced:
```

```python
# Matching Parentheses using stack

s = input("Enter an expression: ")
stack = []
balanced = True

for ch in s:
    if ch == '(':
        stack.append(ch)

    elif ch == ')':
        if len(stack) == 0:
            balanced = False
            break
        else:
            stack.pop()

if len(stack) != 0:
    balanced = False
if balanced:
    print("Parentheses are balanced")
else:
    print("Parentheses are not balanced")
```

OUTPUT :

```
neDrive/Desktop/Assigmnet/Drishti_Ds_lab/Ds_stack.py
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter element: 3
Element pushed
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 2
Deleted element: 3
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 4
Enter a string: stack
Reverse string: kcats
Enter an expression: 8 * x
Parentheses are balanced
```

Experiment- 12 :

Diagonal and Non – diagonal element

Drishti_Ds_lab > Diagnol_Non_Diagnol.py > ...

```python
n = int(input("Enter the matrix size : "))

a = []

print("Enter the matrix elements:")
for i in range(n):
    row = []
    for j in range(n):
        row.append(int(input()))
    a.append(row)

diagonal = 0
non_diagonal = 0

for i in range(n):
    for j in range(n):
        if i == j:
            diagonal = diagonal + a[i][j]
        else:
            non_diagonal = non_diagonal + a[i][j]

print("The sum of diagonal elements:", diagonal)
print("The sum of non-diagonal elements:", non_diagonal)
```

Output :

```
Enter the matrix size : 2
Enter the matrix elements:
2
3
3
4
The sum of diagonal elements: 6
The sum of non-diagonal elements: 6
PS C:\Users\DELL\OneDrive\Desktop\Assigmnet>
```